QUESTION-1:

Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

PROGRAM:

```
 #include<stdio.h>

#include<unistd.h>

 int main() { }

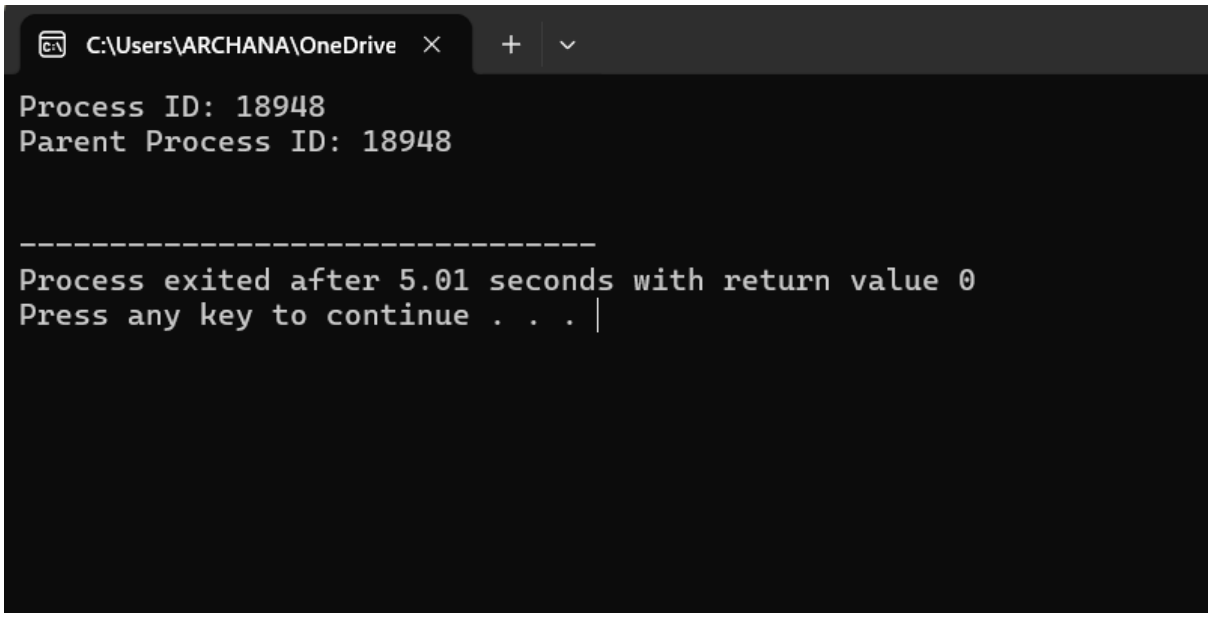
printf("Process ID: %d\n", getpid() );

 printf("Parent Process ID: %d\n", getpid() );

 return 0;

}
```

OUTPUT:



QUESTION-2:

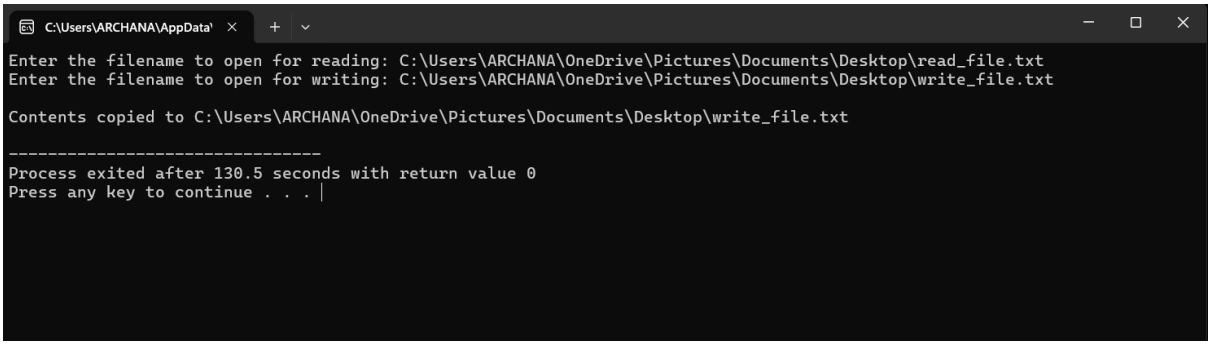
Identify the system calls to copy the content of one file to another and illustrate the same using a C program.

PROGRAM:

```
#include <stdio.h>
#include <stdio.h>
#include <stdlib.h
int main() {
    FILE *fptr1, *fptr2;
    char src[200], dest[200], c;
    printf("Enter the filename to open for reading: ");
    scanf("%s", src)
    fptr1 = fopen(src, "r");   // read mode
    if (fptr1 == NULL) {
        printf("Cannot open file %s\n", src);
        exit(0);
    }
    printf("Enter the filename to open for writing: ");
    scanf("%s", dest);
    fptr2 = fopen(dest, "w");  // write mode
    if (fptr2 == NULL) {
        printf("Cannot open file %s\n", dest);
        fclose(fptr1);
        exit(0);
    }
    while ((c = fgetc(fptr1)) != EOF) {
        fputc(c, fptr2);
    }

    printf("\nContents copied to %s\n", dest);
    fclose(fptr1);
    fclose(fptr2);
    return 0;
}
```

OUTPUT:



QUESTION-3:

Design a CPU scheduling program with C using First ComeFirst Served technique with the following considerations.

a. All processes are activated at time 0.

b. Assume that no process waits on I/O devices.

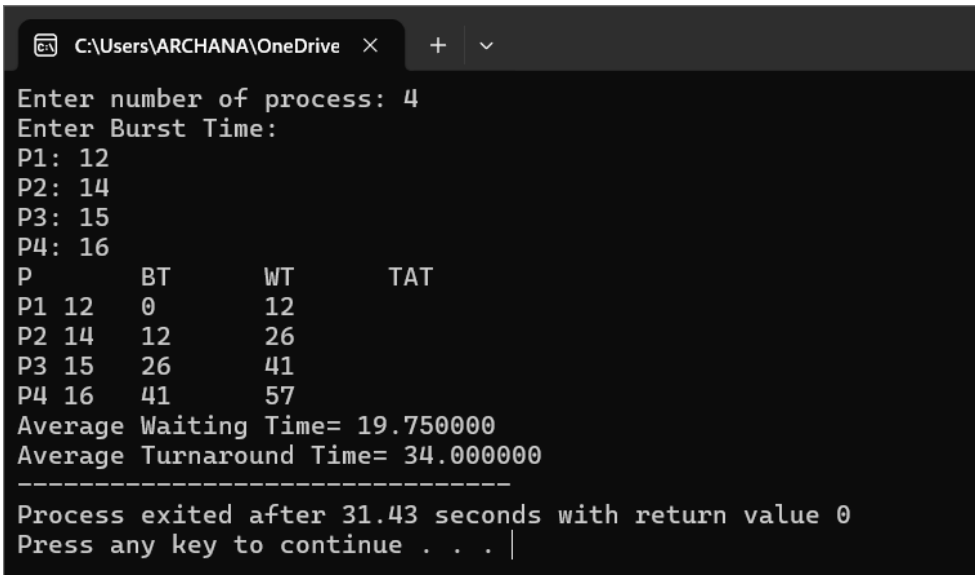
PROGRAM:

```
#include <stdio.h>
int main() {
int A[100][4];
int i, j, n, total = 0, index, temp;
float avg_wt, avg_tat;
printf("Enter number of process: ");
scanf("%d", &n);
printf("Enter Burst Time:\n");
for (i = 0; i < n; i++) {
printf("P%d: ", i + 1);
scanf("%d", &A[i][1]);
A[i][0] = i + 1;
}
for (i = 0; i < n; i++) {
index = i;
```

```
for (j = i + 1; j < n; j++)
if (A[j][1] < A[index][1])
index = j;
temp = A[i][1];
A[i][1] = A[index][1];

A[index][1] = temp;
temp = A[i][0];
A[i][0] = A[index][0];
A[index][0] = temp; }
A[0][2] = 0;
for (i = 1; i < n; i++) { A[i][2] = 0; for (j = 0; j < i; j++)
A[i][2] += A[j][1]; total += A[i][2];
} avg_wt = (float)total / n; total = 0;
        printf("P        BT      WT      TAT\n");
 for (i = 0; i < n; i++) {
A[i][3] = A[i][1] + A[i][2]; total += A[i][3];
        printf("P%d %d          %d      %d\n", A[i][0],A[i][1],
A[i][2], A[i][3]); }
avg_tat = (float)total / n;
printf("Average Waiting Time= %f", avg_wt); printf("\nAverage Turnaround Time= %f", avg_tat);
}
```

OUTPUT:



QUESTION-4:

Construct a scheduling program with C that selects the waiting process with the smallest execution time to execute next.

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

#include<ctype.h>

int isKeyword(char buffer[]){ char keywords[32][10] =

{"main","auto","break","case","char","const","continue","default",

"do","double","else","enum","extern","float","for","goto",

"if","int","long","register","return","short","signed",

"sizeof","static","struct","switch","typedef", "unsigned","void","printf","while"}; int i, flag = 0;

for(i = 0; i < 32; ++i)

{

if(strcmp(keywords[i], buffer) == 0)

{

flag = 1; break;

}

}

return flag;

}

int main()

{

char ch, buffer[15], operators[] = "+-*/%="; FILE *fp; int i,j=0; fp = fopen("C:/Users/Sahasra/OneDrive/Desktop/flex_input.txt","r"); if(fp == NULL){ printf("error while opening the file\n"); exit(0);

}

while((ch = fgetc(fp)) != EOF){    for(i = 0; i < 6; ++i){    if(ch == operators[i])    printf("%c is operator\n", ch);

  }
```

```
  if(isalnum(ch)){    buffer[j++] = ch;
  }
  else if((ch == ' ' || ch == '\n') && (j != 0)){    buffer[j] = '\0';    j = 0;
  if(isKeyword(buffer) == 1)    printf("%s is keyword\n", buffer);
  else
  printf("%s is identifier\n", buffer);
  }

}
fclose(fp); return 0;
}
```

OUTPUT:

